

Build/Boost/Buy Decision Tree

Quick reference for deciding whether to build, improve (boost), or buy/integrate an existing solution.

The Three Options

BUILD = Write it yourself

BOOST = Improve something that exists (yours or open source)

BUY = Use an existing commercial or open-source solution

Decision Tree

```
START: Do we need this at all?  
→ If NO → Don't build anything. Document why.  
→ If YES → Does a solution already exist?  
  → If NO → Go to BUILD  
  → If YES → Is it good enough?  
    → If YES → Go to BUY  
    → If NO → Can we fix/improve it?  
      → If YES → Go to BOOST  
      → If NO → Go to BUILD
```

Decision Factors

GO TO BUILD IF:

- Existing solution doesn't handle your specific constraint (example: "AI can't directly mutate State")

- You're testing a hypothesis about how something should work (example: "Does authority invariant matter?")
- The problem is novel enough that generalizing would destroy the specificity (example: "Billing requires Tier 2 approval")
- Cost of integration (not cost of building) is higher than cost of building
- You need strong control over the implementation (especially for security/compliance)
- The custom version will teach you something about the problem that a generic version wouldn't

GO TO BOOST IF:

- The open-source or internal solution is 80%+ correct
- The missing 20% is fixable without major refactor
- You can contribute the fix upstream (good citizenship, future maintenance)
- The improvement generalizes (other people will benefit, so it's not just selfish)
- The boost effort is <20% of the build effort

GO TO BUY IF:

- The existing solution handles 90%+ of your use case
 - The remaining 10% is acceptable, workaround-able, or low-priority
 - Maintenance cost of keeping the solution up-to-date is acceptable
 - Vendor risk is low (not a single-person project, reasonable funding, clear roadmap)
 - Integration cost is reasonable
 - You're willing to accept the vendor's architectural decisions (even if not ideal)
-

Examples from Paige's Work

State Database Abstraction (BUILD)

Why not BUY? - Existing solutions (SQLAlchemy, Django ORM) exist
- But they make architectural choices that would require reworking State's transaction model - State needs explicit row-level locking, custom conflict detection, immutability enforcement - Buying would mean accepting architectural decisions designed for web apps, not audit trails

Verdict: Build custom db.py layer. Added 300 lines of code but gave full control over transaction semantics.

State API Framework (BUY/BOOST)

Why not BUILD? - FastAPI already exists and solves ~95% of the problem - The 5% (structured output validation) is handled by application code, not framework code - Maintenance burden of FastAPI is externalized

Verdict: Buy FastAPI. Build custom interpretation_pipeline.py on top. Clear separation: buy infrastructure, build domain logic.

Meridian Knowledge Base (BOOST, then BUILD)

Scenario: The KB is 40% stale.

Option A (BUILD): Build a freshness-detection system for knowledge base articles.

- Cost: ~5 days - Benefit: Accurate freshness tracking - Risk: Maintenance burden

Option B (BOOST): Audit and fix the top 20% of articles (high-traffic, high-value).

- Cost: ~2 days - Benefit: Most readers see fresh content - Risk: The bottom 80% is still stale (but lower traffic)

Verdict: BOOST (fix top 20%), then BUILD light (AI spot-checks citations).

Pragmatic tradeoff: covers the high-impact problem while the KB cleanup continues in parallel.

State vs. Manual CURRENT_STATE.md (BOOST vs. BUILD)

Scenario: Current system is manual. Should we build State?

Option A (BOOST): Improve the existing manual process. - Add checklists to the CURRENT_STATE.md file - Add timestamps - Add version numbers - Document what "consequential" means - Cost: ~1 day - Benefit: Slightly more structure to manual process

Option B (BUILD): Build State. - Cost: ~2 weeks - Benefit: Automatic interpretation, less manual maintenance, atomic transitions, audit trail

Verdict: BUILD.

The manual process teaches you what structure you need, but building it in software catches things (concurrent edits, silent conflicts, accidental overwrites) that manual discipline can't prevent.

Common Traps

"We should build this because it's unique to us."

→ Being unique is not a reason to build. Low integration effort is. Clear boundary with existing systems is.

"This open-source project hasn't been maintained in 6 months."

→ Find out if it's dead (no one cares anymore) or stable (it works, nothing needs changing). These are different.

"The commercial solution costs \$X/month. For that, we could build it."

→ You could. But would you? And would you maintain it in 2 years? Commercial solutions get security updates. Your code doesn't.

"We need to build it because our use case is special."

→ Most use cases feel special up close. Check if the "special" part is actually load-bearing (does it affect the core problem?) or just different.

Decision Template

When facing a build/boost/buy question:

1. **Define the problem, not the solution.** (Avoid: "Should we build a context maintenance tool?" Ask: "How do we prevent users from losing track of what's true?")
2. **List three options for that problem.** (Manual process + checklists, buy existing tool, build State-like tool)
3. **For each option, estimate:**
 - Development time (if building)
 - Integration time (if buying)
 - Maintenance burden (ongoing)
 - Vendor risk (if buying)
 - Scope creep risk (if building)
4. **Pick the option with the lowest total cost.** (Total = dev + integration + maintenance, weighted by certainty of estimate)
5. **Document why you didn't pick the others.** (This is important for future you, when you second-guess the decision.)

Questions to Ask

Before Building: - If I stop working on this, can someone else maintain it? - What would I need to change if requirements shift? (How brittle is this design?) - Can I test this assumption without building the full thing?

Before Boosting: - Will my fix benefit other users, or is it too specific to my use case? - Will I stay in sync with upstream changes?

Before Buying: - What happens if the vendor disappears? - How hard is it to leave if we need to? - What architectural constraints am I accepting?

The Rule of Thumb

Build the hard part. Buy/boost the infrastructure.

State = Built the hard part (authority invariant, transaction semantics). Bought the infrastructure (FastAPI, Postgres).

Meridian = Would have bought the NLP/classification (cloud APIs exist). Built the discovery process.

This isn't absolute, but it's a good starting heuristic.